ЛАБОРАТОРНА РОБОТА № 1
Асинхронна обробка даних у TypeScript

Мета: Налаштувати середовище Node.js і проєкт TypeScript, набути практичних навичок роботи з Promise та async/await. Реалізувати паралельне завантаження профілів, повторні спроби після помилок, послідовну обробку партій і обмеження часу виконання асинхронних операцій.

Хід роботи:

1 Перевірка Node.js і npm

На робочому комп'ютері Node.js уже встановлено. Для перевірки доступності середовища та менеджера пакетів у PowerShell виконано наведені команди.

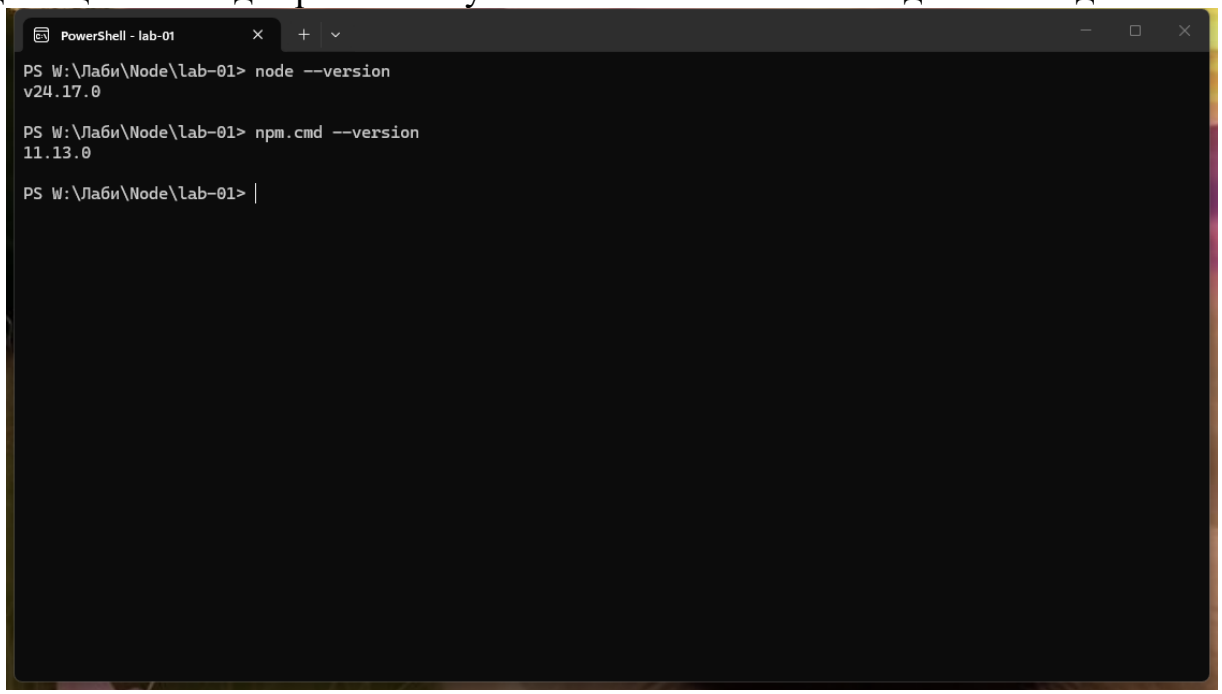


Рис. 1. Перевірка версій Node.js та npm у PowerShell

Команди вивели v24.17.0 та 11.13.0 відповідно. Це підтверджує, що обидва інструменти доступні з командного рядка. Node.js виконує JavaScript поза браузером, а npm установлює залежності та запускає сценарії проєкту.
У Windows використано npm.cmd та npx.cmd — командні файли тих самих npm і npx. Це дозволяє запускати їх без зміни політики виконання сценаріїв PowerShell.
Гілка Node.js 24 належить до LTS. Якщо середовища немає на іншому комп'ютері, його потрібно встановити з офіційного сайту nodejs.org, відкрити нове вікно PowerShell і повторити перевірку версій.

2 Ініціалізація проєкту та встановлення залежностей

У папці lab-01 виконано npm init -y. Параметр -y приймає початкові значення без діалогу та створює package.json.

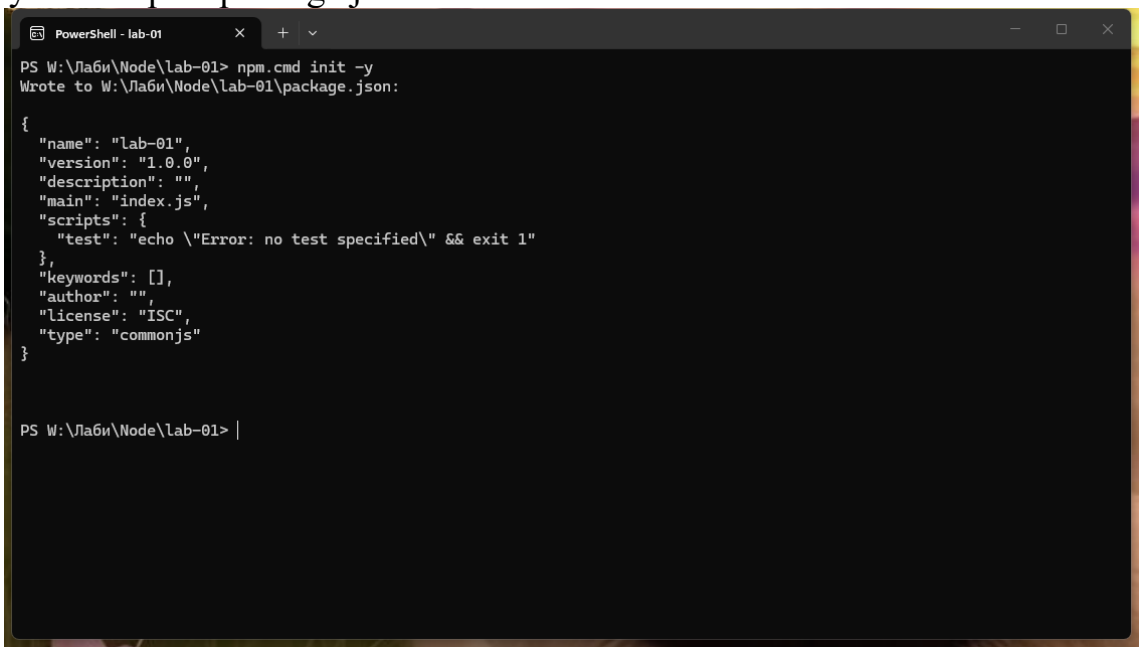


Рис. 2. Створення package.json командою npm init -y

TypeScript установлено як залежність розробки. Пакет @types/node додає описи типів для process, таймерів та інших можливостей Node.js.

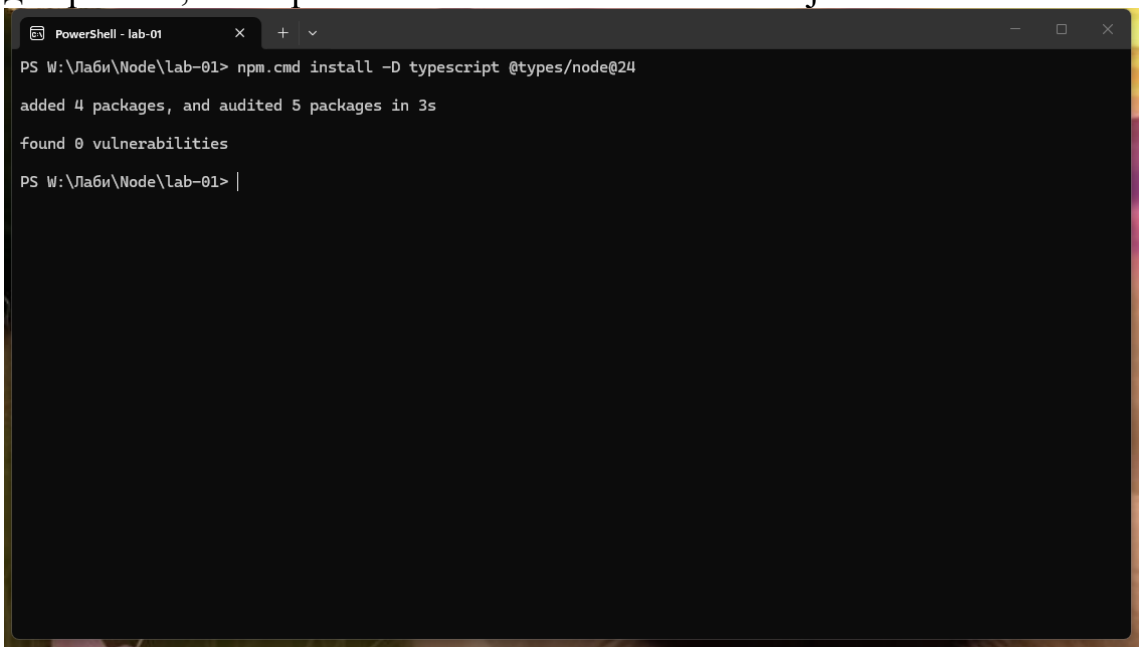


Рис. 3. Установлення TypeScript і типів Node.js

3 Створення конфігурації TypeScript

Команда npx tsc --init створила tsconfig.json. Також створено папку src для вихідних файлів та перевірено версію компілятора.

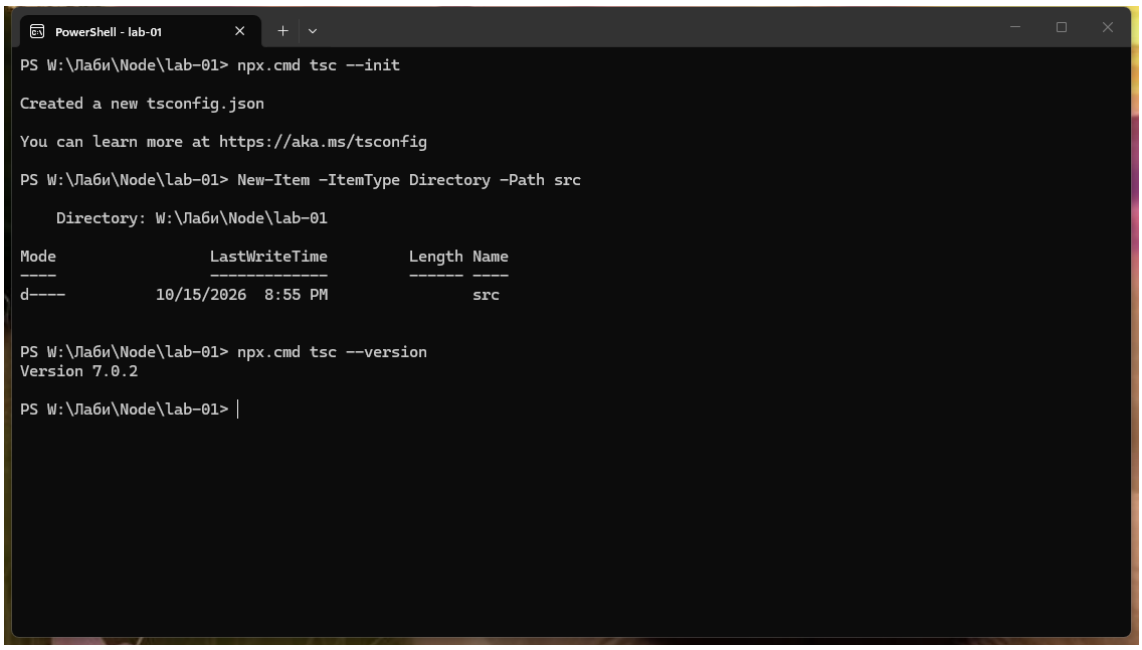


Рис. 4. Створення tsconfig.json, папки src та перевірка tsc

Після створення налаштовано tsconfig.json. Вихідні файли розміщуються у src, а скомпільований JavaScript — у dist.

Лістинг:

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "rootDir": "src",
    "outDir": "dist",
    "types": ["node"],
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "noEmitOnError": true,
    "forceConsistentCasingInFileNames": true,
    "skipLibCheck": true
  },
  "include": ["src/**/*.ts"],
  "exclude": ["node_modules", "dist"]
}
```

4 Параметри компіляції та сценарії npm

Параметр	Призначення
target: ES2022	Версія синтаксису згенерованого JavaScript.
module і moduleResolution: NodeNext	Правила модулів та пошуку імпортів для Node.js.
rootDir: src; outDir: dist	Розділення вихідного TypeScript і результатів компіляції.
strict: true	Суворі перевірки типів.
noUncheckedIndexedAccess	Урахування undefined під час доступу за індексом.
noEmitOnError: true	Заборона створення JavaScript за наявності помилок компіляції.
types: node	Підключення описів типів Node.js.

У package.json указано type: module та сценарії build, start і test. Build запускає tsc; start виконує скомпільований main.js; test спочатку компілює проєкт, а потім запускає перевірки.

Лістинг:

```
{
  "name": "lab-01",
  "version": "1.0.0",
  "description": "Лабораторна робота 1. Асинхронна обробка даних у TypeScript",
  "private": true,
  "main": "dist/async-utils.js",
  "scripts": {
    "build": "tsc",
    "start": "node dist/main.js",
    "test": "npm run build && node --test --test-reporter=spec test/async-utils.test.mjs"
  },
  "keywords": [],
  "author": "Сокирко Владислав Володимирович, ІПЗ-23-4",
  "license": "ISC",
  "type": "module",
  "devDependencies": {
    "@types/node": "^24.19.2",
    "typescript": "^7.0.2"
  }
}
```

5 Структура програми та компіляція

Усі функції та інтерфейс UserProfile експортовано з одного файлу src/async-utils.ts. Файл src/main.ts імпортує їх і запускає демонстраційні сценарії. Розширення .js у шляху імпорту відповідає файлу після компіляції.

Лістинг:

```
import {
  delay, fetchUserProfiles, retryOperation, processInBatches,
  raceWithTimeout, type UserProfile,
} from './async-utils.js';
```

Команда npm run build завершилася без помилок. У папці dist створено async-utils.js та main.js.

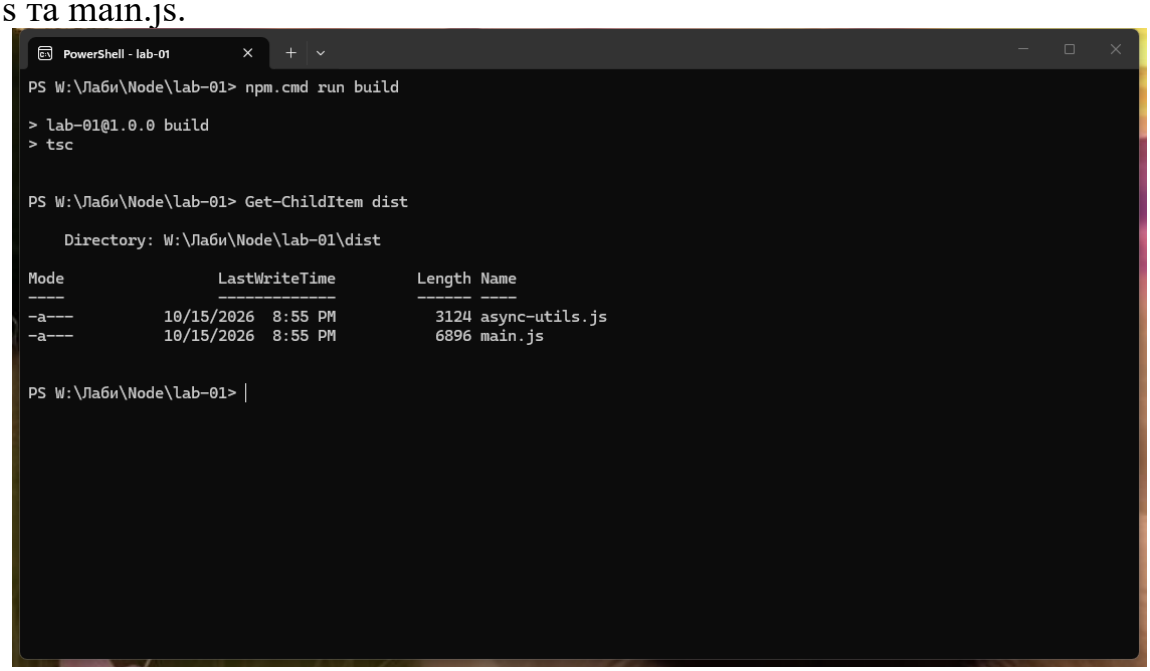


Рис. 5. Успішна компіляція та перевірка файлів dist у PowerShell

6 Реалізація функції delay

Завдання: створити Promise<void>, який завершується після заданої затримки. Усередині new Promise таймер setTimeout викликає resolve. Await очікує завершення Promise, не блокуючи потік JavaScript.

Лістинг:

```
export async function delay(ms: number): Promise<void> {
  if (!Number.isFinite(ms) || ms < 0 || ms > 2_147_483_647) {
    throw new RangeError('ms must be between 0 and 2147483647');
  }

  await new Promise<void>((resolve): void => {
    setTimeout(resolve, ms);
  });
}
```

Перевірка аргументу є додатковим захистом: від'ємні, нескінченні та надто великі значення відхиляються через RangeError. Для коректного аргументу до setTimeout передається саме ms.

7 Перевірка функції delay

node dist/main.js delay

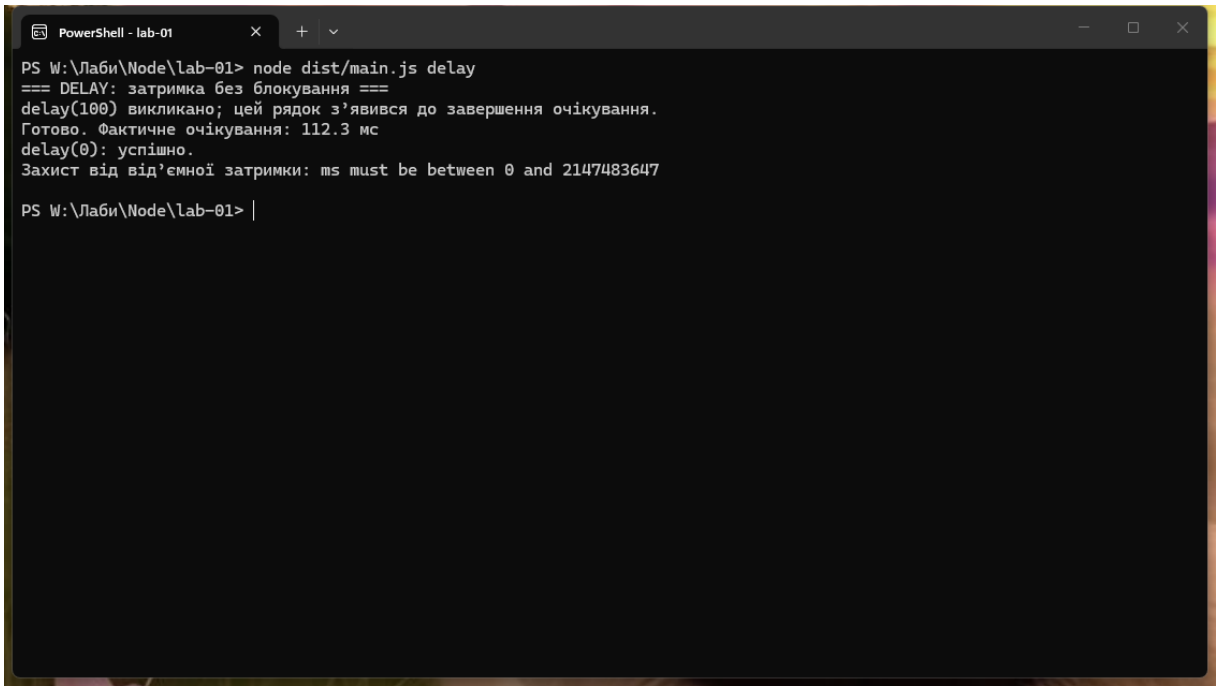


Рис. 6. Виконання delay та перевірка некоректного аргументу

Демонстрація спочатку викликає delay(100), зберігає Promise у pending та одразу виводить повідомлення. Лише після await pending з'являється рядок «Готово». Це показує, що виклик delay сам по собі не зупиняє виконання наступного синхронного коду.

Перевірка	**Очікування**	**Фактичний результат**
delay(100)	Успішне завершення після очікування	112,3 мс у зафіксованому запуску
delay(0)	Коректне асинхронне завершення	Успішно
delay(-1)	Відхилення некоректного аргументу	RangeError перехоплено

Значення 100 мс задає затримку таймера, але не гарантує виконання callback точно в цю мить. На фактичний час впливають планування операційною системою та завантаженість циклу подій. Тому результат 112,3 мс не означає, що у функцію потрібно додавати штучну поправку.

Promise має три основні стани: pending, fulfilled і rejected. У цьому прикладі resolve переводить Promise у fulfilled, а помилка перевірки аргументу спричиняє rejected.

8 Реалізація fetchUserProfiles

Завдання: паралельно отримати профілі для всіх userIds. Метод map одразу створює асинхронні операції. Кожна очікує випадкову затримку 50–150 мс і формує id, name та email. Promise.all очікує всі результати.

Лістинг:

```
export interface UserProfile {
  id: string;
  name: string;
  email: string;
}

export async function fetchUserProfiles(
  userIds: string[],
): Promise<UserProfile[]> {
  const requests: Promise<UserProfile>[] = userIds.map(
    async (id: string): Promise<UserProfile> => {
      const delayMs: number = Math.floor(Math.random() * 101) + 50;
      await delay(delayMs);

      return {
        id,
        name: `User ${id}`,
        email: `user${id}@example.com`,
      };
    },
  );

  return await Promise.all(requests);
}
```

9 Перевірка паралельного завантаження

node dist/main.js profiles

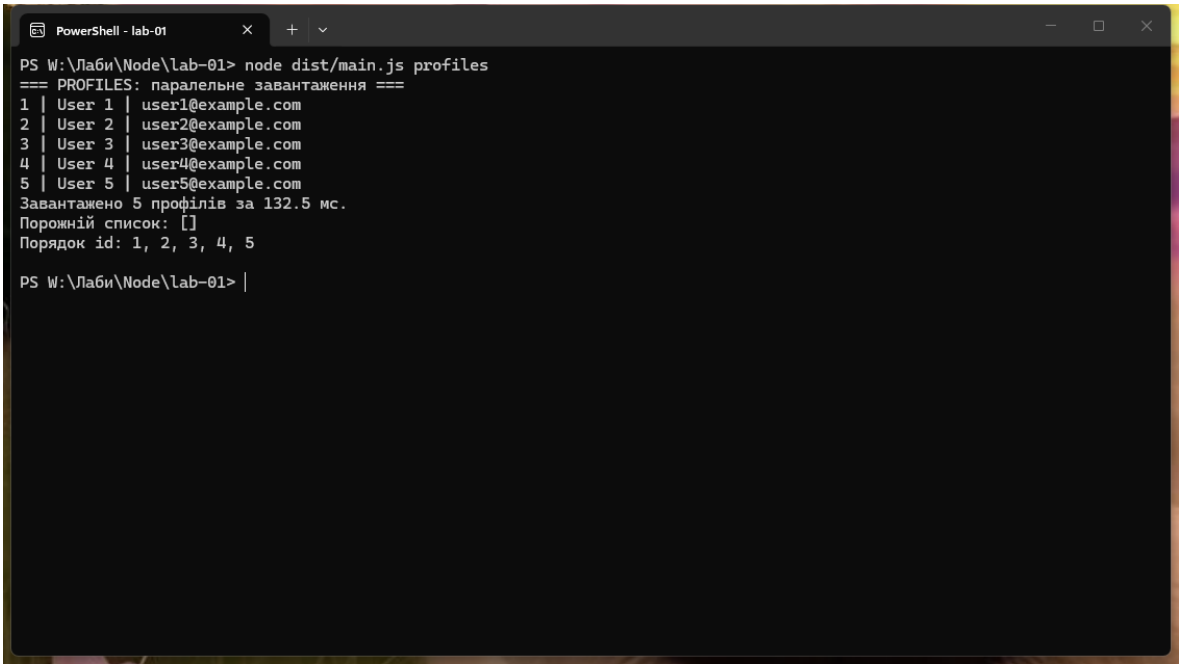


Рис. 7. П’ять завантажених профілів, час виконання та порожній список

Для ідентифікаторів 1, 2, 3, 4, 5 отримано п’ять профілів. Імена мають вигляд User N, а адреси — userN@example.com. Це демонстраційні дані: реальних мережевих запитів у функції немає.

У зафіксованому запуску всі профілі отримано за 132,5 мс. За послідовного виконання лише сума п’яти мінімальних затримок становила б 250 мс. При паралельному запуску очікування наближається до часу найповільнішого запиту з урахуванням накладних витрат.

Порядок результатів Promise.all відповідає порядку вхідних Promise, навіть якщо окремі операції завершуються в іншому порядку. Порожній список дає порожній масив без окремого циклу обробки.

Властивість	**Як перевірено**
Кількість і поля профілів	Вивід п’яти записів та автоматичне порівняння об’єктів.
Порожній список	fetchUserProfiles([]) повертає [].
Паралельний старт	Тест фіксує запуск усіх таймерів до завершення першого.
Порядок результатів	Тест завершує операції у зворотному порядку, але перевіряє початковий порядок id.

Якщо одна операція у Promise.all відхиляється, сукупний Promise також відхиляється. Інші вже запущені операції автоматично не скасовуються.

10 Реалізація retryOperation

Завдання: повторювати операцію після помилки. За умовою maxRetries означає загальну кількість спроб, за замовчуванням 3. Після успіху повертається результат; між невдалими спробами виконується delay(100).

Лістинг:

```
export async function retryOperation(
  operation: () => Promise<unknown>,
  maxRetries: number = 3,
): Promise<unknown> {
  if (!Number.isInteger(maxRetries) || maxRetries < 1) {
    throw new RangeError('maxRetries must be a positive integer');
  }

  for (let attempt: number = 1; ; attempt++) {
    console.log(`Спроба ${attempt}...`);
    try {
      return await operation();
    } catch (error: unknown) {
      if (attempt === maxRetries) {
        throw error;
      }
      await delay(100);
    }
  }

}
```

11 Перевірка повторних спроб

node dist/main.js retry

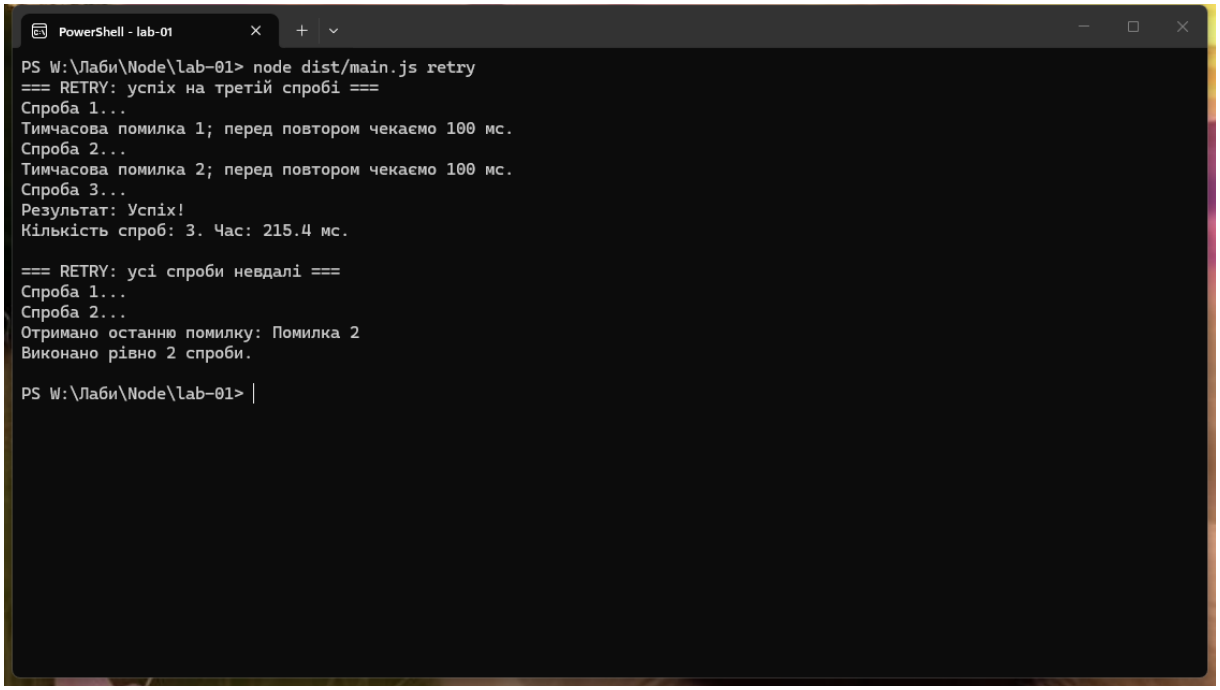


Рис. 8. Успіх на третій спробі та передавання останньої помилки

У першому сценарії операція двічі викидає тимчасову помилку, а на третій спробі повертає «Успіх!». Загальний час 215,4 мс узгоджується з двома паузами по 100 мс та накладними витратами.

Спро-ба	**Результат операції**	**Подальша дія**
1	Тимчасова помилка 1	Пауза 100 мс
2	Тимчасова помилка 2	Пауза 100 мс
3	Успіх!	Повернення результату без нової паузи

У другому сценарії обидві дозволені спроби завершуються помилкою. Функція передає назовні саме останню помилку — «Помилка 2». Після останньої спроби повторення та додаткова затримка не виконуються.

Операція має тип () => Promise<unknown>, оскільки може повертати значення різних типів. Тип unknown вимагає перевірки перед специфічними діями над результатом. Значення maxRetries = 0 або дробове значення відхиляються до першого виклику операції.

12 Реалізація processInBatches

Завдання: розділити масив на партії та обробити їх послідовно. Slice створює чергову партію, а await processor(batch) не дозволяє почати наступну до завершення поточної. Обробляються всі партії, включно з першою.

Лістинг:

```
export async function processInBatches(
  items: number[],
  batchSize: number,
  processor: (batch: number[]) => Promise<number[]>,
): Promise<number[]> {
  if (!Number.isInteger(batchSize) || batchSize < 1) {
    throw new RangeError('batchSize must be a positive integer');
  }

  const results: number[] = [];
  const total: number = Math.ceil(items.length / batchSize);

  for (let offset: number = 0; offset < items.length; offset += batchSize) {
    const batchNumber: number = Math.floor(offset / batchSize) + 1;
    const batch: number[] = items.slice(offset, offset + batchSize);
    console.log(`Обробка партії ${batchNumber}/${total}...`);

    const processed: number[] = await processor(batch);
    for (const value of processed) {
      results.push(value);
    }
  }

  return results;
}
```

13 Перевірка обробки партіями

node dist/main.js batches

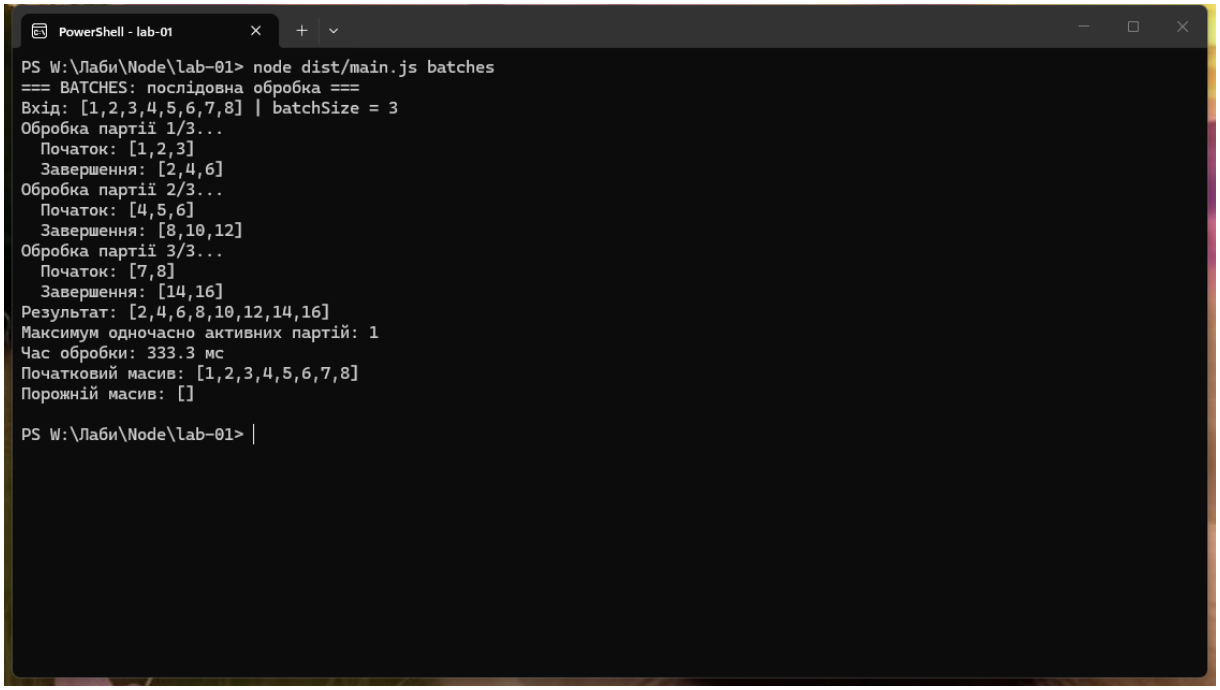


Рис. 9. Порядок обробки партій та підсумковий масив

Партія	Вхідні дані	Результат множення на 2
1 із 3	[1, 2, 3]	[2, 4, 6]
2 із 3	[4, 5, 6]	[8, 10, 12]
3 із 3	[7, 8]	[14, 16]

Остання партія містить два елементи, оскільки довжина масиву 8 не ділиться на розмір партії 3 без остачі. Кількість партій обчислюється як Math.ceil(8 / 3) = 3. Підсумковий результат: [2, 4, 6, 8, 10, 12, 14, 16]. Максимальна кількість одночасно активних processor дорівнює 1. Час 333,3 мс відповідає трьом послідовним очікуванням по 100 мс із накладними витратами.

Початковий масив залишився без змін. Для порожнього масиву функція повертає [] і не викликає processor. Якщо processor завершується помилкою, обробка наступних партій припиняється, а помилка передається викликачеві.

14 Реалізація raceWithTimeout

Завдання: обмежити час очікування переданого Promise. Створюється другий Promise, який відхиляється після timeoutMs. Promise.race повертає стан того учасника, який завершився першим. Власний таймер очищається у finally.

Лістинг:

```
export async function raceWithTimeout(
  promise: Promise<unknown>,
  timeoutMs: number,
): Promise<unknown> {
  if (!Number.isFinite(timeoutMs) || timeoutMs < 0 || timeoutMs > 2_147_483_647) {
    throw new RangeError('timeoutMs must be between 0 and 2147483647');
  }

  let timer: ReturnType<typeof setTimeout> | undefined;
  const timeout: Promise<never> = new Promise<never>((_resolve, reject): void => {
    timer = setTimeout((): void => {
      reject(new Error(`Operation timed out after ${timeoutMs}ms`));
    }, timeoutMs);
  });

  try {
    return await Promise.race([promise, timeout]);
  } finally {
    clearTimeout(timer);
  }
}
```

15 Перевірка обмеження часу

node dist/main.js timeout

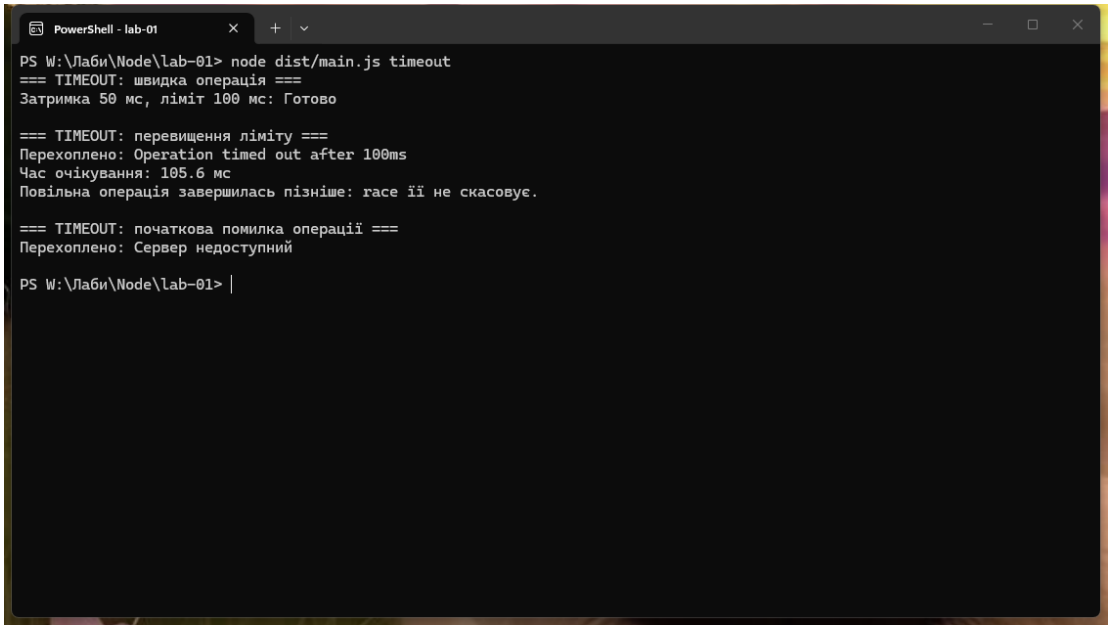


Рис. 10. Швидка операція, таймаут і початкова помилка операції

Перевірено три ситуації. Операція із затримкою 50 мс встигає до ліміту 100 мс і повертає «Готово». Операція із затримкою 200 мс перевищує цей ліміт; отримано повідомлення Operation timed out after 100ms. Час очікування у знімку становить 105,6 мс.

Якщо початковий Promise відхиляється раніше за таймер, назовні передається початкова помилка. У прикладі це «Сервер недоступний», а не повідомлення про таймаут.

```
try {
  await raceWithTimeout(delay(200), 100);
} catch (error: unknown) {
  if (error instanceof Error) {
    console.error(error.message);
  }
}
```

Promise.race обмежує очікування, але сам по собі не скасовує повільну операцію. Це видно з пізнішого повідомлення про її завершення. Для справжнього скасування потрібна підтримка механізму скасування самою операцією.
ClearTimeout у finally прибирає таймер обмеження після завершення race. Це запобігає зайвому очікуванню таймера, коли основна операція вже повернула результат або помилку.

16 Обробка помилок та граничних випадків

Крім прикладів з методички, перевірено некоректні аргументи та помилку processor. Вони допомагають відрізнити контрольоване завершення від зависання або прихованого продовження обробки.

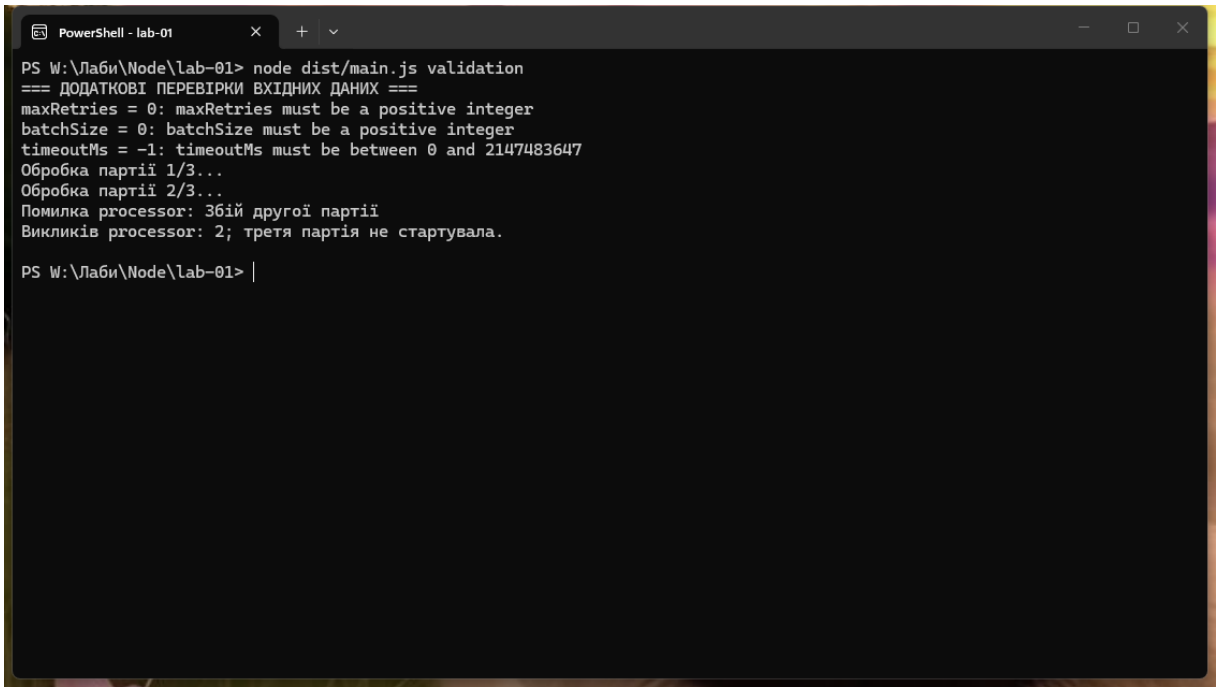


Рис. 11. Перехоплення помилок аргументів і зупинка після збою партії

Вхідна ситуація	**Поведінка**
maxRetries = 0	RangeError; operation не викликається.
batchSize = 0	RangeError; цикл обробки не запускається.
timeoutMs = -1	RangeError; таймер обмеження не створюється.

Вхідна ситуація	Поведінка
Помилка у другій партії	Назовні передано помилку; третя партія не запускається.
Порожні масиви	Повертається порожній результат.
Остання неповна партія	Усі залишкові елементи обробляються один раз.

У catch використано unknown та перевірку instanceof Error перед доступом до message. Непередбачені помилки головної програми обробляються через main().catch(...), після чого process.exitCode установлюється в 1.

17 Демонстраційна CLI програма

Main.ts підтримує запуск усіх прикладів або одного обраного сценарію. Без аргументів використовується all. Передане значення читається з process.argv і зіставляється з таблицею функцій.

Лістинг:

```
async function main(): Promise<void> {
  const demos: Record<string, () => Promise<void>> = {
    delay: demoDelay, profiles: demoProfiles, retry: demoRetry,
    batches: demoBatches, timeout: demoTimeout, validation: demoValidation,
  };
  const selected: string = process.argv[2] ?? 'all';
  if (selected === 'all') {
    for (const demo of Object.values(demos)) {
      await demo();
      console.log('');
    }
  } else {
    const demo: (() => Promise<void>) | undefined = demos[selected];
    if (demo === undefined) {
      throw new Error(`Невідомий сценарій: ${selected}. Доступні:
${Object.keys(demos).join(', ')}, all`);
    }
    await demo();
  }
}

main().catch((error: unknown): void => {
  console.error('Непередбачена помилка:', message(error));
  process.exitCode = 1;
});
```

Команда після компіляції	Що виконується
npm.cmd start	Усі демонстрації послідовно.
node dist/main.js delay	Затримка та перевірка аргументу.
node dist/main.js profiles	Паралельне завантаження профілів.

Команда після компіляції	Що виконується
node dist/main.js retry	Успішні та невдалі повторні спроби.
node dist/main.js batches	Послідовна обробка масиву.
node dist/main.js timeout	Результат, таймаут і початкова помилка.
node dist/main.js validation	Додаткові помилкові сценарії.

Після змін у файлах .ts слід повторно виконати npm run build. Інакше команда node dist/main.js запустить попередню скомпільовану версію.

18 Автоматична перевірка

Тести розміщено у test/async-utils.test.mjs. Для запуску використано вбудований модуль node:test та перевірки node:assert/strict. Команда npm test спочатку компілює проєкт.

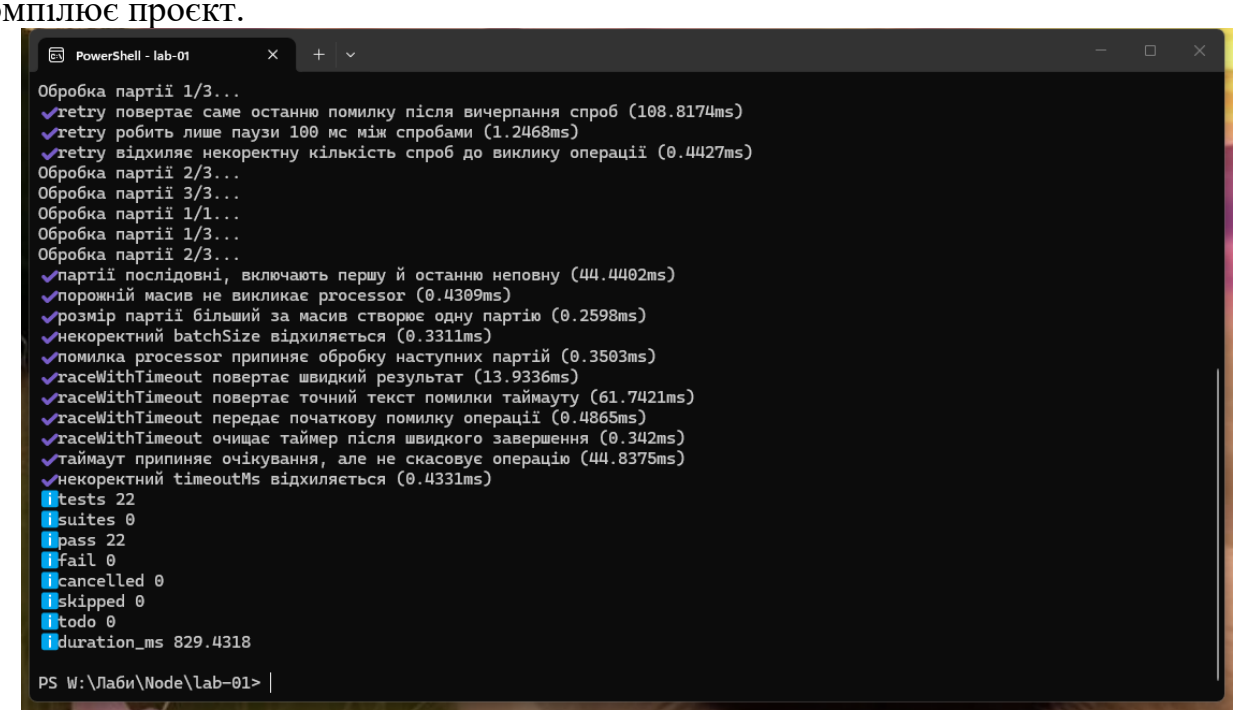


Рис. 12. Підсумок запуску тестів у PowerShell

Група	Тестів	Що перевіряється
delay	3	Асинхронність, нуль, некоректні затримки.
fetchUserProfiles	3	Поля, порядок, порожній список, паралельність і діапазон затримок.
retryOperation	5	Ранній успіх, 3 спроби, остання помилка, паузи, валідація.

Група	Тестів	Що перевіряється
processInBatches	5	Послідовність, усі партії, порожній список, великий розмір, помилки.
raceWithTimeout	6	Успіх, таймаут, початкова помилка, очищення, відсутність скасування, валідація.

Підсумок: tests 22, pass 22, fail 0, skipped 0. У зафіксованому запуску тривалість тестового набору становила близько 829 мс. Це час тестування, а не гарантована характеристика швидкодії програми.

Для перевірки паралельного старту та точного інтервалу повторів у двох тестах контрольовано підмінюються таймери. Так перевіряється логіка планування без залежності від швидкості комп’ютера. Інші сценарії також перевіряють роботу зі справжніми асинхронними операціями.

19 Аналіз результатів та висновки

Сценарій	Зафіксований результат	Пояснення
delay(100)	112,3 мс	Таймер із затримкою 100 мс та накладні витрати.
5 профілів	132,5 мс	Запити запускаються паралельно.
Retry з успіхом на спробі 3	215,4 мс	Дві паузи по 100 мс.
3 партії	333,3 мс	Три послідовні паузи по 100 мс.
Таймаут 100 мс	105,6 мс	Обмеження очікування спрацювало.
Автоматичні тести	22 із 22 успішні	Помилок у перевірених сценаріях не виявлено.

Висновок: Під час виконання роботи налаштовано TypeScript-проєкт у середовищі Node.js та реалізовано п’ять асинхронних функцій. Використано явні типи параметрів і результатів, async/await, Promise.all, Promise.race, try/catch та finally. Паралельне завантаження дозволяє очікувати незалежні операції одночасно. Послідовна обробка партій, навпаки, зберігає контрольований порядок і не допускає одночасного виконання кількох processor. Механізм retry обмежує кількість спроб, а raceWithTimeout обмежує очікування результату. Перевірка успішних сценаріїв, помилок і граничних значень підтвердила правильність реалізації у межах 22 тестів. Фактичний час виконання залежить від планування таймерів та може відрізнятися між запусками.